

DATA STRUCTURES AND REASON FOR THEIR USE

1. Hash Tables / Dictionaries (Key-Value Store)

- **Description:** A data structure mapping unique keys directly to stored values using hash functions.
- **Justification:** In application memory, active vehicles are tracked using plate_number as the **Key** and entry_time as the **Value**. This guarantees $O(1)$ **constant-time performance** for check-ins, existence checks, and checkouts, avoiding costly $O(n)$ linear array searches as vehicle volume grows .

2. In-Memory Cache Store (Redis)

- **Description:** High-speed, in-memory data store supporting key-value string structures, atomic operations, and time-to-live (TTL) expiry.
- **Justification:** Sitting in front of the primary database, Redis stores transient slot counts. Single-command atomic decrements (DECR) and increments (INCR) handle rapid concurrent arrivals at entry gates safely, while caching slot availability prevents database query overhead during peak hours.

3. Schema Validation Models (Pydantic / DTOs)

- **Description:** Typed data transfer objects that define formal data contracts at API boundaries.
- **Justification:** Validates and sanitizes incoming HTTP JSON payloads (e.g., verifying plate_number format and vehicle_type values) before hitting backend logic, returning automatic error responses for malformed data.

4. Relational Entity Tables (SQL Database Relations)

- **Description:** Structured table schemas defined by typed columns, primary keys, foreign key constraints, and relational mappings .
- **Justification:** Ensures long-term data persistence and enforces **ACID properties** (Atomicity, Consistency, Isolation, Durability) for transaction logs, financial auditing, and session histories.